



1 Text-based Image Retrieval

Did you know that you can quite easily extend your implemented search engine to also search for images?

In this text-based retrieval of images, images are (like normal HTML/Text documents) represented by their textual features, that is, the text that is on the website where the image is presented. Obviously, one should avoid considering all terms in the entire document as good features for a certain image. Only terms in the vicinity of an image should be taken as features that describe the image. With vicinity, ideally, the closeness of the terms to the actual image in terms of positions in the rendered HTML document is considered. In this assignment, you should modify your indexer to also index images contained in the crawled pages. More specifically, your task is as follows:

- Extend your database schema to allow storing images.
- Simply consider the vicinity of an image to be the text surrounding the image. For instance, see the illustration on the right-hand side.
- Consider exponential distribution in your scoring model for features(terms) of an image to capture the decreasing importance of terms that appear far from the image. Avoid the use of simple scoring model like *tf*.
- Tune your feature (term) selection according to the parameter you have used in the scoring model.
- You can also make use of HTML tags (such as `<title>`, `<meta>` etc.) for building features of the image.
- In such cases, use a weighting model to incorporate all additional features into the scoring model.
- Design and implement an HTML user interface for image search and add it to your existing search engine.

Don't read this Don't read this Don't read this Don't read this Don't read this Don't read this Don't read this. But here it is written that this nice image is an image of a green Christmas tree with tons of colorful decoration.



Source *openclipart.org*

This image of a Christmas tree was taken from openclipart.org and depicts a very standard, green Christmas tree; there are also Christmas trees in different colors, but this one is green. Don't read this Don't read this Don't read this Don't read this Don't read this Don't read this Don't read this.

Hints

- Exponential distribution: https://en.wikipedia.org/wiki/Exponential_distribution

¹All illustrations on this page are taken from openclipart.org

Questions

- How do you select the window size for feature selection?
- What kind of weighing model have you used in your implementation and why ?

2 Snippets for Enhanced User Satisfaction

It is much nicer if users do not just see the query results in form of urls only, but can also see the context in which the query terms appear in the resulting document. Thus, when presenting the user with the query results, show also a snippet of text that underpins that (hopefully) the document behind the url is really a good match for the query. For instance, this is an excerpt of the Google answer for the query `lgis is-project` constrained to matches of the domain `uni-kl.de`:

Lehrgebiet Informationssysteme: IS Project

www.lgis.informatik.uni-kl.de/cms/courses/is-project/ ▾

The project is going to be offered again in Wintersemester 2015/16. The website of Prof. Michel's group, that hosts this project, moved to dbis.informatik.uni-kl.de ...

IS Project - Databases and Information Systems Group

dbis.informatik.uni-kl.de/index.php/en/teaching/winter-2015.../is-project ▾

Information Systems (IS) Project 2015/16. In this project, we consider building a full-fledged Web Search engine. Details on the registration procedure can be ...

IS Project 2014/15

dbis.informatik.uni-kl.de/index.php/en/teaching/.../is-project-2014-15 ▾

Information Systems (IS) Project WS 14/15. In this new project, being offered in Wintersemester 2014/15 for the first time, we consider building a full-fledged Web ...

One common solution in literature is to collect at crawling time information on where each term occurs and to pre-process snippets. Instead, simply keep at crawling/indexing time the entire text of the website and post-process it to find a snippet of text that suits nicely the user query.

In this assignment, your task is to implement the creation of such snippets and to show them in your HTML-based user interface.

- Adapt the indexer/crawler and the db schema in a way that can keep also the textual content of a page.
- The snippet should be quite short, like in the above illustration. More precisely, a snippet should contain maximum 32 terms.
- Have in mind that, as in the above example, in disjunctive mode not all documents contain all of the query terms. For the terms not included in the document, mention the missing term in your UI.
- Even if all terms appear in the documents, it does not have to be the case that they can all be found within 32 contiguous terms. In such cases, it is possible to present a snippet which is created by concatenation of smaller sub-snippets separated by "...". Sub-snippets should contain at least 8 words in it.
- Implement a scoring model for snippets considering the query-term frequency and coverage of query terms in a snippet. Present the best one to the user which should give maximum coverage as well as maximize the query term frequency for the snippet according to your scoring model.

- Keep in mind that your scoring model should capture the number of sub-snippets.

Hints

- Implement the snippets only in your HTML user interface. Do not show snippets in the CLI or JSON interface.

Questions

- How is the performance of the query answering affected? Why?
- For a given query and a document, more than one snippet can be found to be presented to the user. How do you choose the snippet that is shown to the user? Why?

3 Query Expansion

In this assignment, you will enrich the search mechanism by introducing the tilde ($\sim$) operator to your search engine. The idea is to take into consideration also semantically related meanings (e.g., synonyms) for terms that are prefixed by the tilde operator. For instance, a query $\sim holiday$ would also allow documents containing *vacation* to qualify for the query results. In a simple way, we can solve this problem by searching the documents of presence of any synonym for those query terms.

Your task is as follows:

- Load the *WordNet* dictionary to collect synonyms of English query terms using the extjwnl Java library.
- You should extract synonyms for all possible positions (e.g., NOUN, VERB, etc. according to WordNet) of a query term as WordNet suggests.
- For German query terms, you should use the text file given in <https://www.openthesaurus.de/about/download>. You do not need to consider the words written in brackets for German terms in that file.
- Expand the user query in conjunctive normal form (CNF) with the retrieved synonyms for the query terms marked with $\sim$. A query $\sim a \ b \ c \ \sim d$ is getting expanded into $(a \text{ OR } a_1 \text{ OR } a_2 \text{ OR } a_3 \text{ OR } \dots) \text{ AND } b \text{ AND } c \text{ AND } (d \text{ OR } d_1 \text{ OR } d_2 \text{ OR } \dots)$, where the terms a_i and d_j are the expansion terms for term a and d , respectively.

Hints

- Description of extjwnl Java library <https://github.com/extjwnl/extjwnl>. Follow the instructions in the README file and the Documentation for including the library and the WordNet dictionary in your project.
- There are lots of other libraries available to work with WordNet. If you have already used some other library, feel free to use that.

4 Shingling and Near-Duplicate Detection

Once your crawled index grows, you will sooner or later face the problem of having duplicate websites in your index. If query results contain many duplicates, it might drastically limit the chance of a user to find good results—besides wasting storage space on your hard disk. With near duplicates, we refer to documents that are not exact copies but slightly differ in their textual content. Still, it is important to identify them. The methods you are implementing in this task can further be used to identify plagiarism in documents such as PhD theses or any other publication.

4.1 Shingling

In this assignment you are supposed to implement shingling, a way to represent a text document by a set of word sequences.

Shingling computes for each document the set of sequences of k words that are contained in the document. We define the set of k -shingles of a document d as the set of word sequences of size k words that are contained in the document. For instance, for a document [a b b c], where a, b, and c are words, we have the following set of 2-shingles that represents the document: {ab, bb, bc}. The k -shingles of a document d_i are denoted as $S_k(d_i)$.

Your task is as follows:

- Compute for each document that you have indexed—now as you have also the full-text content that you used for the snippets already—the set of k -shingles.
- Parameterize your entire code to allow changing k (no hard coding!). For this assignment, use $k = 4$, which is a typical number for the size of shingles.
- Store all shingles of a document in a table inside your database.
- In order to identify near-duplicate documents, you can compare the shingles of documents. Documents that share many shingles are likely to be near-duplicates.
- Implement a brute-force duplicate detection algorithm that computes the Jaccard measure between each pair of documents. For this, take the set of shingles for a document d_1 and a document d_2 , and compute the Jaccard coefficient that is the ratio of the number shingles that both documents share (i.e., the intersection) and the number of shingles in any of the documents (i.e., the union):

$$J(S(d_1), S(d_2)) = \frac{|S(d_1) \cap S(d_2)|}{|S(d_1) \cup S(d_2)|}$$

- You might want to limit the number of documents for which you do the pairwise computation of the Jaccard measure, but compute it for at least 10 000 pairs.
- Clearly, a Jaccard value of 1 means that the documents have identical sets of shingles, while a value of 0 tells us that both documents have not even a single shingle in common.
- Write SQL queries that compute for each pair of documents their Jaccard value, store this in another table inside the DB.
- Create a user defined function (UDF) that returns for a given document d_i and similarity threshold θ a table that contains documents d_j with $J(S(d_i), S(d_j)) \geq \theta$.
- Think about creating indices on the table containing the shingles in order to compute the pairwise Jaccard value efficiently wherever possible/meaningful.

4.2 Min-Hashing for Efficient Similarity Search

Instead of the above brute-force variant of computing the Jaccard coefficient between all pairs of sets, there is a much more efficient, though approximate, algorithm, the so called min-hashing approach. The key idea is to apply a hash function on each of a document's shingles. The hash function maps a shingle to an integer value, for instance by applying the MD5 (or SHA-1) hash function on the string of words of a shingle and interpreting some of the bytes given by MD5 (or SHA-1) as an integer.

In the simplest case, for each document, the smallest value of such hash values is kept. Hence, each document is represented by one such integer. The key point is that the probability that for two documents the two min-hash-values are identical, written as $Pr[\min(d_1) = \min(d_2)]$ stands in direct relationship to the Jaccard similarity, as follows:

$$Pr[\min(d_1) = \min(d_2)] = \frac{|S(d_1) \cap S(d_2)|}{|S(d_1) \cup S(d_2)|}$$

This is a very crude approximation in practice: if two documents share the same min-value, we would conclude that the Jaccard value is 1.0. On the other hand, this can be computed very efficiently: looking up all documents that have the same min-value.

In practice, one does not consider only the minimum of such hash values, but the n smallest values. Alternatively, one can take the minimum value for n different hash functions. In any case, one would then approximate the Jaccard coefficient by the fraction of min-values (out of n) on which document d_1 and d_2 agree.

Your task is the following,

- Implement min-hashing with configurable number of min-values n over the shingles created above.
- Devise SQL queries that compute the approximated Jaccard coefficient by using min-hashing. Postgres has a built-in function called MD5 that computes the MD5 hashcode for an input string and it is also possible to convert the bytes obtained through MD5 into an integer.
- Compute how well the approximated value resembles the actual Jaccard coefficient. For this, use the following number for the parameter n : $n = 1, 4, 16, 32$. For each value, compute for at least 10 000 pairs of documents the estimated Jaccard value using min-hashing and compare it to the exact Jaccard value computed above.
- For these comparisons, report on the average, the median, and the first and third quartile of the observed absolute error between the actual Jaccard coefficient and the estimated one.
- Obviously, you should use the same document pairs as for the actual Jaccard above.
- Create a pdf document containing a table with the above measures (obviously, for varying n). You might also add a plot (e.g., using the Gnuplot tool). You can submit this document by storing it in your git repository or uploading it in the Mattermost channel of the group.

Hints

- Min-Hashing explained on Wikipedia: <http://en.wikipedia.org/wiki/MinHash>
- Bit more explanation on shingling and hashing <http://nlp.stanford.edu/IR-book/html/htmledition/near-duplicates-and-shingling-1.html>
- Implement only a minimum of code in Java (e.g., the parsing of the HTML documents into shingles). Once the shingles are in the DB, let the DB do the job by using UDFs and queries via JDBC.